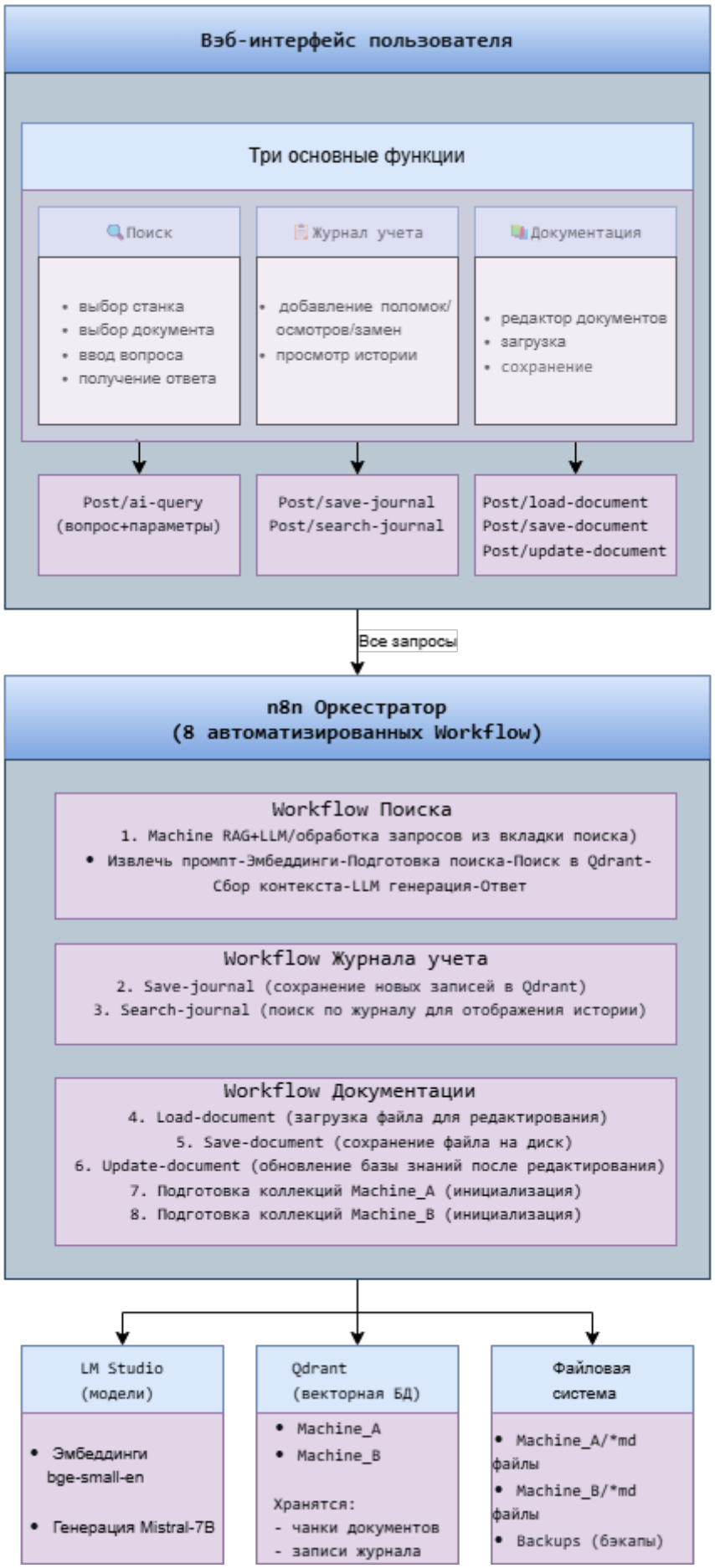
## Архитектура



**Оркестратор (n8n) – 8 Workflow, управляющих логикой**:

- Загрузка документации в Qdrant (2 однотипных workflow, для 2 видов станков)

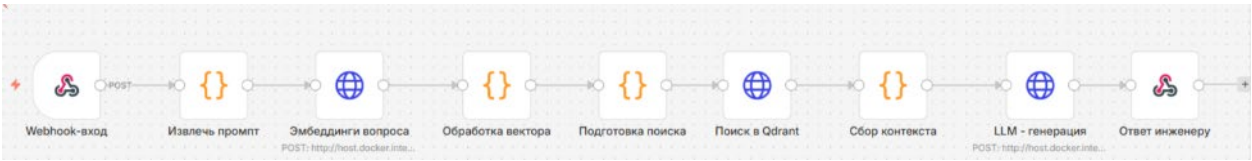



Workflow загружает и индексирует .md файлы документации, преобразуя их в векторные точки для поиска:

1. When clicking ‘Execute workflow’ – ручной триггер, запускающий процесс индексации.
2. Read/Write Files from Disk – читает все .md файлы из директории /home/node/.n8n-files/documents/machine_A/.
3. Extract from File – извлекает текстовое содержимое из каждого файла, сохраняя метаданные (имя файла, путь).
4. Code in JavaScript3 (Обогащение метаданными) – определяет для каждого файла:
   - Станок (machine_A или machine_B) по пути или содержимому
   - Тип документа (certificates, error_codes, maintenance, manual) по имени файла Добавляет поля source, machine_id, doc_type, filePath, tenant_id.
5. Code in JavaScript (Чанкование) – разбивает текст на фрагменты по 1000 символов с перекрытием 200 символов для сохранения контекста между чанками. Каждый чанк получает свой порядковый индекс.
6. HTTP Request (Эмбеддинги) – отправляет каждый чанк в LM Studio для получения векторного представления (эмбеддинга) через модель bge-small-en-v1.5.
7. Code in JavaScript2 (Объединение) – сопоставляет каждый чанк с его эмбеддингом, объединяя текст, метаданные и вектор в единый объект.
8. Code in JavaScript1 (Подготовка точек) – формирует из каждого чанка точку для Qdrant с уникальным ID (на основе времени и случайного числа), вектором и полным payload.
9. HTTP Request1 (Загрузка в Qdrant) – отправляет подготовленные точки в коллекцию Machine_A через PUT-запрос к Qdrant.

Результат: все .md файлы из указанной директории разбиты на чанки, векторизованы и загружены в Qdrant для последующего семантического поиска.

- Основной поиск (RAG)



Логика работы Workflow «MACHINE RAG+LLM»

Workflow автоматически обрабатывает запросы пользователя по цепочке из 9 узлов:

1. Webhook-вход – принимает POST-запрос от интерфейса с полями: станок, тип документа, вопрос.
2. Извлечь промпт – парсит запрос, выделяет prompt, machine_id, doc_type для передачи дальше.
3. Эмбеддинги вопроса – отправляет текст в LM Studio, получает векторное представление (эмбеддинг).
4. Обработка вектора – объединяет полученный вектор с исходными данными о станке и типе документа.
5. Подготовка поиска – формирует запрос к Qdrant: коллекция по станку, фильтр только по machine_id (без типа документа), лимит 100 записей.
6. Поиск в Qdrant – выполняет поиск в векторной БД, получает все похожие чанки для данного станка.
7. Сбор контекста – фильтрует результаты по типу документа, сортирует по дате, формирует структурированный текстовый контекст.
8. LLM – генерация – отправляет контекст и вопрос в LM Studio, модель Mistral-7B генерирует ответ строго по контексту.
9. Ответ инженеру – возвращает сгенерированный ответ в интерфейс.

Ключевая особенность: фильтрация по типу документа выполняется в узле сбора контекста, а не в Qdrant, что упростило архитектуру и обеспечило гибкость обработки всех типов документов.

- Загрузка и обновление документации

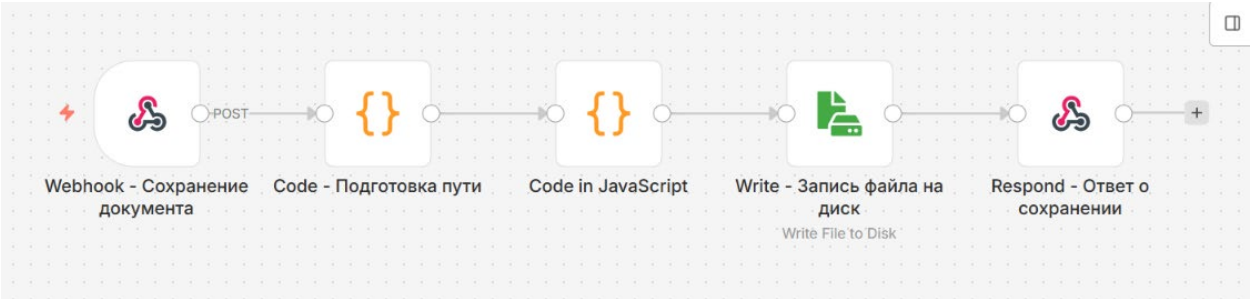



Логика работы workflow «Загрузка документа в редактор».

Workflow загружает содержимое выбранного документа для отображения в редакторе интерфейса:

1. Webhook – принимает POST-запрос от интерфейса с параметрами fileName и machine_id.
2. Code in JavaScript1 – формирует полный путь к файлу: /home/node/.n8n-files/documents/machine_{machineId}/{fileName} и передает его дальше.
3. Read/Write Files from Disk – читает файл по указанному пути.
4. Extract from File – извлекает текстовое содержимое из прочитанного файла.
5. Respond to Webhook – возвращает содержимое файла в интерфейс вместе с fileName и machine_id. Если файл не найден, возвращает заглушку # Файл не найден.

Результат: документ загружается в редактор интерфейса для просмотра и редактирования.




Логика работы Workflow «Сохранение документа на диск»

Workflow сохраняет отредактированный документ из интерфейса в файловую систему:

1. Webhook – Сохранение документа – принимает POST-запрос от интерфейса с параметрами fileName, machine_id и content (текст документа).
2. Code – Подготовка пути – формирует полный путь для сохранения: /home/node/.n8n-files/documents/machine_{machineId}/{fileName} и проверяет наличие контента.
3. Code in JavaScript – преобразует текстовый контент в бинарный формат (Buffer) для записи, сохраняя метаданные файла.
4. Write – Запись файла на диск – выполняет физическую запись файла по указанному пути.
5. Respond – Ответ о сохранении – возвращает в интерфейс подтверждение об успешном сохранении с данными файла.

Результат: отредактированный документ сохраняется на диске, заменяя предыдущую версию.



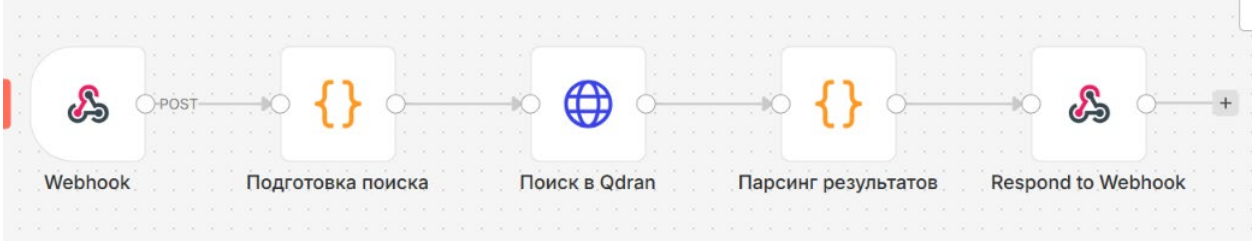
Логика работы Workflow «Обновление документа в базе знаний»

Workflow выполняет полное обновление документа в базе знаний после редактирования: удаляет старые вектора, перечитывает файл и загружает новые точки в Qdrant.

1. Webhook – Обновление документа – принимает POST-запрос от интерфейса с параметрами fileName, machine_id и опционально content (если документ сохранялся ранее).
2. Code – Подготовка к удалению – определяет тип операции:
   - Если есть content – режим сохранения файла (переход к сохранению)
   - Если content отсутствует – режим обновления базы знаний: создает фильтр для удаления всех точек, связанных с данным файлом
3. Code in JavaScript (Создание backup) – перед удалением запрашивает из Qdrant все старые точки этого файла и сохраняет их в JSON-файл в директории /backups на случай необходимости восстановления.

4. HTTP – Удаление из Qdrant – отправляет POST-запрос в Qdrant на удаление всех точек с указанным source (именем файла) и machine_id.
5. Read – Чтение файла с диска – читает актуальную версию файла из файловой системы.
6. Extract – Извлечение текста – извлекает текстовое содержимое из прочитанного файла.
7. Code – Обогащение метаданными – определяет станок и тип документа по имени файла и пути, добавляет source, machine_id, doc_type, filePath.
8. Code – Чанкинг документа – разбивает текст на фрагменты по 1000 символов с перекрытием 200 символов для сохранения контекста.
9. HTTP – Эмбеддинги – отправляет каждый чанк в LM Studio для получения векторного представления.
10. Code – Объединение чанков – сопоставляет каждый чанк с его эмбеддингом.
11. Code – Подготовка points – формирует из каждого чанка точку для Qdrant с уникальным ID (хеш от имени файла и индекса), вектором и полным payload.
12. HTTP – Загрузка в Qdrant – отправляет подготовленные точки в соответствующую коллекцию.
13. Respond – Ответ об обновлении – возвращает в интерфейс подтверждение об успешном обновлении с данными файла.
Результат: документ полностью переиндексирован в Qdrant с актуальным содержимым, старые точки удалены, создан backup предыдущей версии.

- Сохранение записей журнала

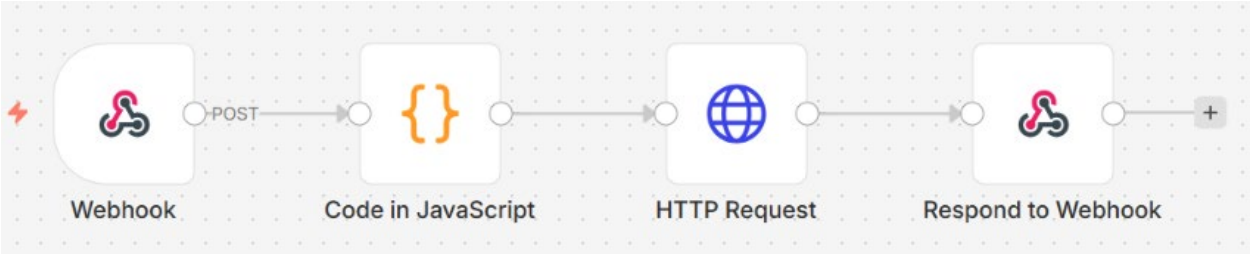



Логика работы Workflow «Поиск по журналу учета»
Workflow возвращает все записи журнала для отображения в интерфейсе (вкладка «Журнал учета»):
1. Webhook – принимает POST-запрос от интерфейса с параметром machine_id.
2. Подготовка поиска – формирует запрос к Qdrant:
- Определяет коллекцию (Machine_A или Machine_B) по machine_id
- Создает нулевой вектор (поиск всех точек без семантического ранжирования)
- Устанавливает фильтр только по machine_id (без фильтрации по типу записей)
- Лимит – 100 записей
3. Поиск в Qdrant – отправляет запрос в Qdrant, получает все точки выбранного станка.
4. Парсинг результатов – обрабатывает полученные данные:
- Извлекает id, score и payload для каждой точки

- Сортирует записи по дате (новые сверху)

5. Respond to Webhook – возвращает в интерфейс массив точек с полными данными для отображения в таблицах журнала.

Результат: интерфейс получает все записи журнала для выбранного станка, отсортированные по дате, и отображает их в соответствующих вкладках (поломки, осмотры, замены).




Логика работы Workflow «Сохранение данных в журнал учета»

Workflow сохраняет новые записи из журнала учета (поломки, осмотры, замены) в векторную базу данных Qdrant:

1. Webhook – принимает POST-запрос от интерфейса с данными:
   - machine_id – идентификатор станка
   - timestamp – время сохранения
   - breakdowns, inspections, replacements – массивы записей соответствующих типов
2. Code in JavaScript (Подготовка точек) – выполняет следующие операции:
   - Генерирует уникальный UUID для каждой записи
   - Создает нулевой вектор размерностью 384 (временное решение, эмбеддинги будут добавлены позже)
   - Для каждой поломки формирует точку с полями: machine_id, type='breakdown', date, code, description, cause, solution, worker, timestamp, text
   - Для каждого осмотра формирует точку с полями: machine_id, type='inspection', date, inspection_type, item, result, notes, worker, timestamp, text
   - Для каждой замены формирует точку с полями: machine_id, type='replacement', date, item, part_type, hours, next, worker, timestamp, text
   - Определяет имя коллекции (Machine_A или Machine_B) на основе machine_id
3. HTTP Request (Загрузка в Qdrant) – отправляет PUT-запрос в Qdrant с массивом подготовленных точек для добавления в соответствующую коллекцию.
4. Respond to Webhook – возвращает в интерфейс подтверждение об успешном сохранении с данными о количестве добавленных записей.

Результат: все новые записи журнала добавляются в Qdrant с нулевыми векторами (позже будут пересчитаны) и становятся доступными для поиска по журналу.

**Потоки данных:**

- Пользовательский запрос: Интерфейс → Webhook (n8n) → извлечение промпта → получение эмбеддинга → поиск в Qdrant (без фильтра по типу, только по станку) → пост-

фильтрация по типу документа → формирование контекста → вызов LLM → генерация ответа → отображение.
- Загрузка документации: Ручной запуск в n8n → чтение .md файлов → извлечение текста → чанкование → эмбеддинги → загрузка в Qdrant.
- Добавление записи в журнал: Интерфейс (журнал) → Webhook `/save-journal` → подготовка точки (с нулевым вектором) → загрузка в Qdrant (позже вектора будут пересчитаны).
- Обновление документа: Интерфейс (редактор) → Webhook `/update-document` → удаление старых точек → сохранение нового файла → перезаливка с новыми эмбеддингами.

**Модель / LLM**

Выбранный подход: RAG (Retrieval-Augmented Generation) с использованием локальных моделей через LM Studio.

Почему локально:

- Конфиденциальность данных (вся информация остаётся внутри периметра предприятия).
- Независимость от внешних API и интернета.
- Возможность тонкой настройки под специфику.

Модели:

- Эмбеддинговая: `text-embedding-bge-small-en-v1.5` (размерность 384, отлично подходит для технических текстов).
- Генеративная: `mistralai/mistral-7b-instruct-v0.3` – идеальна для работы с инструкциями и релевантным поиском.

Параметры генерации:

- `temperature = 0.1` – минимизация случайности, строго по контексту.
- `max_tokens = 500` – достаточная длина для развёрнутого ответа.
- Системный промпт: требует отвечать только на основе предоставленного контекста, без выдумывания.